

LAPORAN TUGAS BESAR 1

IF2123 Aljabar Linier dan Geometri

Pustaka Aljabar Linier dalam Java: Sistem Persamaan Linier, Determinan, Matriks Balikan, Interpolasi, dan Regresi Spline

Kelompok Gerobak Pasar Kosambi



Anggota Kelompok

Jeremy Gerald Sutanto	13525104
Christian Immanuel	13525116
Denzel Santoso	13525014

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
Semester I Tahun 2026/2027

Daftar Isi

1	Pendahuluan	1
1.1	Deskripsi Masalah	1
1.2	Latar Belakang	1
1.3	Tujuan	1
2	Dasar Teori	2
2.1	Sistem Persamaan Linier	2
2.1.1	Eliminasi Gauss	2
2.1.2	Eliminasi Gauss-Jordan	2
2.1.3	Metode Matriks Balikan	2
2.1.4	Kaidah Cramer	2
2.2	Determinan Matriks	2
2.3	Matriks Balikan	3
2.4	Interpolasi Polinomial	3
2.5	Natural Cubic Spline Interpolation	4
2.6	Regresi Spline Kubik	4
3	Implementasi	5
3.1	Struktur Kelas	5
3.2	Arsitektur Pustaka	6
3.3	Rincian Algoritma	6
3.4	Alur Program	7
3.5	Penanganan Kesalahan	8
3.6	Penggunaan Maven dan Pembuatan Berkas JAR	9
4	Eksperimen	9
4.1	Determinan Matriks (Kasus 5.1)	10
4.2	Matriks Balikan (Kasus 5.2)	11
4.3	SPL $A\mathbf{x} = \mathbf{b}$ (Kasus 5.3)	11
4.4	SPL Berbentuk Matriks <i>Augmented</i> (Kasus 5.4)	13
4.5	SPL Berbentuk Umum (Kasus 5.5)	14
4.6	Aplikasi SPL: Regresi Linier Harga Rumah (Kasus 5.6)	15
4.7	Interpolasi Polinomial (Kasus 5.7)	15
4.8	Natural Cubic Spline (Kasus 5.8)	16
4.9	Regresi Spline (Kasus 5.9)	17

4.10 Eksperimen Tambahan	18
5 Penutup	19
5.1 Kesimpulan	19
5.2 Saran	20
5.3 Refleksi Anggota Kelompok	20
Daftar Pustaka	21
Lampiran	21

1 Pendahuluan

1.1 Deskripsi Masalah

Dalam skenario tugas besar ini, seorang peneliti muda bernama Mitya memimpin eksperimen berskala raksasa bernama Proyek Stuzha di Teyvat. Perhitungan berdimensi tinggi yang diperlukan proyek tersebut tidak lagi dapat ditangani oleh kalkulator saku miliknya yang sering *error*. Kelompok kami ditugaskan membangun perangkat lunak sekelas *Photomath* versi Teyvat, yaitu mesin kalkulasi otomatis yang mampu melumat sistem matriks dan persamaan linier yang rumit.

1.2 Latar Belakang

Aljabar linier adalah fondasi komputasi modern yang menggerakkan simulasi rekayasa, pemodelan aliran energi, analisis data, hingga dinamika sistem kompleks. Agar kalkulasi Proyek Stuzha tidak runtuh di tengah jalan, kami ditugaskan merancang dan membangun pustaka (*library*) komputasi aljabar linier mandiri berbasis bahasa pemrograman Java yang modular, tangguh, dan terdokumentasi rapi. Pustaka ini menjadi mesin utama alat komputasi Mitya untuk:

- menyelesaikan sistem persamaan linier (SPL), menghitung determinan, dan mencari matriks balikan guna menganalisis keseimbangan sirkuit daya dan kestabilan instrumen;
- melakukan interpolasi polinomial, *natural cubic spline interpolation*, dan regresi *spline* untuk memetakan jalur transfer energi serta memodelkan kontur data eksperimen secara presisi.

1.3 Tujuan

1. Mengimplementasikan metode penyelesaian SPL, determinan, matriks balikan, interpolasi polinomial, *natural cubic spline interpolation*, dan regresi *spline* secara mandiri menggunakan Java.
2. Membangun pustaka aljabar linier yang terstruktur secara modular, bersih, dan terdokumentasi dengan baik tanpa memakai pustaka perhitungan matriks, regresi, atau interpolasi instan.
3. Menguji keandalan pustaka pada berbagai kasus uji analitis, serta menyusun laporan analisis hasil perhitungannya.

2 Dasar Teori

2.1 Sistem Persamaan Linier

Sistem persamaan linier (SPL) dengan m persamaan dan n variabel dapat ditulis sebagai $A\mathbf{x} = \mathbf{b}$ dengan $A \in \mathbb{R}^{m \times n}$, atau dalam bentuk matriks *augmented* $[A \mid \mathbf{b}]$. Solusi SPL dapat berupa solusi tunggal, tak hingga banyak solusi, atau tidak ada solusi [1].

2.1.1 Eliminasi Gauss

Matriks *augmented* diubah menjadi matriks eselon baris menggunakan operasi baris elementer (OBE): menukar dua baris, mengalikan baris dengan skalar tak nol, dan menambahkan kelipatan suatu baris ke baris lain. Solusi diperoleh dengan substitusi mundur. Untuk menjaga stabilitas numerik dipakai *partial pivoting*, yaitu pada kolom ke- k dipilih baris dengan $|a_{ik}|$ terbesar sebagai pivot sehingga pengali baris selalu bernilai $|l_{ik}| \leq 1$. Setelah eliminasi, SPL tidak memiliki solusi jika terdapat baris berbentuk $[0 \cdots 0 \mid c]$ dengan $c \neq 0$. Jika tidak, banyaknya variabel bebas sama dengan $n - \text{rank}(A)$; bila nol maka solusi tunggal, selain itu solusi berbentuk parametrik.

2.1.2 Eliminasi Gauss-Jordan

Eliminasi dilanjutkan hingga diperoleh matriks eselon baris tereduksi: setiap pivot bernilai 1 dan merupakan satu-satunya elemen tak nol pada kolomnya. Solusi dapat dibaca langsung tanpa substitusi mundur yang panjang.

2.1.3 Metode Matriks Balikan

Jika A persegi dan $\det(A) \neq 0$, maka $A\mathbf{x} = \mathbf{b}$ memiliki solusi tunggal $\mathbf{x} = A^{-1}\mathbf{b}$.

2.1.4 Kaidah Cramer

Untuk SPL persegi dengan $\det(A) \neq 0$, solusi tunggalnya adalah

$$x_i = \frac{\det(A_i)}{\det(A)}, \quad i = 1, \dots, n, \quad (1)$$

dengan A_i adalah matriks A yang kolom ke- i -nya diganti $\mathbf{b}$.

2.2 Determinan Matriks

Determinan $\det(A)$ hanya terdefinisi untuk matriks persegi dan menentukan apakah matriks memiliki balikan ($\det(A) \neq 0$).

Reduksi baris (OBE). Matriks direduksi menjadi segitiga atas. Menambahkan kelipatan satu baris ke baris lain tidak mengubah determinan, setiap pertukaran baris mengalikan determinan dengan -1 , dan mengalikan satu baris dengan k mengalikan determinan dengan k . Bila s adalah banyaknya pertukaran baris dan u_{ii} elemen diagonal hasil reduksi, maka

$$\det(A) = (-1)^s \prod_{i=1}^n u_{ii}. \quad (2)$$

Ekspansi kofaktor. Untuk sembarang baris i (atau kolom j),

$$\det(A) = \sum_{j=1}^n a_{ij} C_{ij}, \quad C_{ij} = (-1)^{i+j} M_{ij}, \quad (3)$$

dengan M_{ij} determinan submatriks hasil penghapusan baris i dan kolom j . Pemilihan baris atau kolom yang memuat banyak nol mengurangi jumlah suku yang harus dihitung. Kompleksitas metode ini $O(n!)$ sehingga hanya praktis untuk matriks kecil, sedangkan reduksi baris berkompleksitas $O(n^3)$.

2.3 Matriks Balikan

Matriks persegi A memiliki balikan A^{-1} dengan $AA^{-1} = A^{-1}A = I$ jika dan hanya jika $\det(A) \neq 0$.

Metode augmentasi $[A \mid I]$. Matriks $[A \mid I]$ direduksi dengan eliminasi Gauss-Jordan menjadi $[I \mid A^{-1}]$.

Metode adjoin.

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A), \quad \text{adj}(A) = C^T, \quad (4)$$

dengan C matriks kofaktor, $C_{ij} = (-1)^{i+j} \det(M_{ij})$.

2.4 Interpolasi Polinomial

Diberikan n titik berbeda (x_i, y_i) , terdapat tepat satu polinom berderajat paling tinggi $n - 1$, $P(x) = a_0 + a_1x + \cdots + a_{n-1}x^{n-1}$, yang melalui seluruh titik tersebut [2]. Koefisien diperoleh dengan menyelesaikan SPL Vandermonde

$$\begin{bmatrix} 1 & x_1 & \cdots & x_1^{n-1} \\ \vdots & \vdots & & \vdots \\ 1 & x_n & \cdots & x_n^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}. \quad (5)$$

Polinom ini dapat dipakai untuk interpolasi (di dalam $[x_{\min}, x_{\max}]$) maupun ekstrapolasi. Matriks Vandermonde berondisi buruk bila n besar, sehingga galat pembulatan dapat membesar.

2.5 Natural Cubic Spline Interpolation

Interpolasi polinomial berderajat tinggi rentan berosilasi liar. *Cubic spline* menghindarinya dengan memakai potongan polinom kubik $S_j(x)$ pada tiap subinterval $[x_j, x_{j+1}]$, $j = 0, \dots, n-2$ untuk n titik, dengan syarat S melalui semua titik serta S , S' , dan S'' kontinu di setiap simpul dalam $[2, 3]$. Pada *natural spline* ditambahkan syarat batas $S''(x_0) = S''(x_{n-1}) = 0$.

Bila $M_j = S''(x_j)$ dan $h_j = x_{j+1} - x_j$, maka tiap segmen berbentuk

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3, \quad (6)$$

dengan

$$a_j = y_j, \quad b_j = \frac{y_{j+1} - y_j}{h_j} - \frac{h_j(2M_j + M_{j+1})}{6}, \quad c_j = \frac{M_j}{2}, \quad d_j = \frac{M_{j+1} - M_j}{6h_j}. \quad (7)$$

Kekontinuan turunan pertama di simpul dalam menghasilkan SPL **tridiagonal** untuk $M_1, \dots, M_{n-2}$:

$$h_{i-1}M_{i-1} + 2(h_{i-1} + h_i)M_i + h_iM_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right), \quad i = 1, \dots, n-2, \quad (8)$$

dengan syarat batas natural $M_0 = M_{n-1} = 0$. Matriks koefisiennya dominan secara diagonal ($2(h_{i-1} + h_i) > h_{i-1} + h_i$) sehingga selalu non-singular dan solusi M_i tunggal.

2.6 Regresi Spline Kubik

Berbeda dengan interpolasi, regresi *spline* tidak mengharuskan kurva melalui setiap titik, tetapi meminimalkan jumlah kuadrat galat terhadap N data pengamatan [4, 5]. Untuk K simpul (*knot*) $\xi_1 < \dots < \xi_K$, ruang *spline* kubik berdimensi $M = 4 + K$ dengan *truncated power basis*

$$h_0 = 1, \quad h_1 = x, \quad h_2 = x^2, \quad h_3 = x^3, \quad h_{3+k}(x) = (x - \xi_k)_+^3, \quad (9)$$

dengan $(u)_+ = \max(0, u)$. Fungsi regresi adalah $\hat{y}(x) = \sum_{j=0}^{3+K} \beta_j h_j(x)$, yang otomatis kontinu hingga turunan kedua (C^2). Dengan matriks desain $X \in \mathbb{R}^{N \times M}$, $X_{ij} = h_j(x_i)$, masalah $X\beta \approx \mathbf{y}$ bersifat *overdetermined* ($N > M$). Solusi kuadrat terkecil diperoleh dari proyeksi ortogonal $\mathbf{y}$ ke $\text{Col}(X)$, yaitu persamaan normal

$$(X^T X) \hat{\beta} = X^T \mathbf{y}. \quad (10)$$

Jika kolom-kolom X bebas linier maka $X^T X$ non-singular dan $\hat{\beta} = (X^T X)^{-1} X^T \mathbf{y}$ tunggal. Persamaan normal ini adalah SPL berukuran $M \times M$ yang diselesaikan dengan eliminasi Gauss.

3 Implementasi

Seluruh pustaka dan program ditulis dalam Java (target Java 17) tanpa pustaka eksternal untuk perhitungan matriks. Pustaka hanya memakai `java.lang.Math` untuk fungsi dasar (`abs`, `max`, `round`) dan kelas standar Java untuk berkas dan masukan (`java.io`, `java.util`).

3.1 Struktur Kelas

Kode sumber berada di `src/main/java/algeo` dan dibagi ke dalam paket-paket berikut.

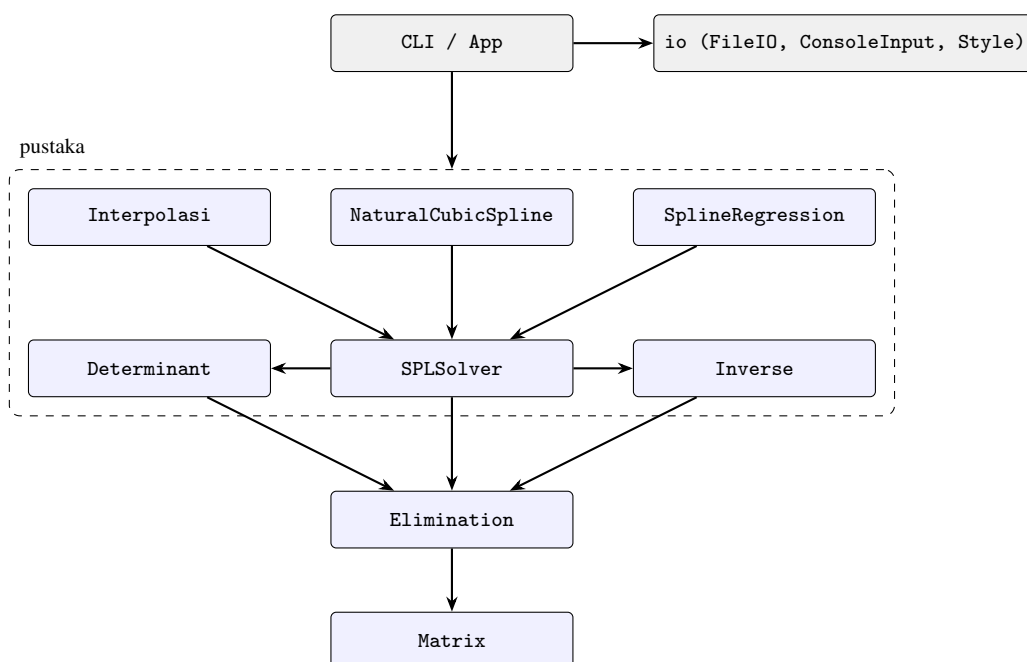
Tabel 1: Paket dan kelas pada program

Paket	Kelas	Tanggung jawab dan metode penting
<code>matrix</code>	<code>Matrix</code>	Struktur data matriks (atribut <code>rows</code> , <code>cols</code> , <code>double[][] data</code>). OBE: <code>swapRows</code> , <code>multiplyRow</code> , <code>addRowMultiple</code> . Operasi: <code>add</code> , <code>subtract</code> , <code>multiply</code> (skalar dan matriks), <code>transpose</code> , <code>submatrix</code> , <code>slice</code> , <code>augment</code> , <code>identity</code> , <code>copy</code> . Pemformatan: <code>round3</code> , <code>formatNumber</code> .
<code>spl</code>	<code>Elimination</code>	<code>toRowEchelonForm</code> dan <code>toReducedRowEchelonForm</code> dengan <i>partial pivoting</i> ; mencatat langkah OBE; <code>tolerance</code> untuk deteksi nol.
	<code>SPLSolver</code>	<code>gauss</code> , <code>gaussJordan</code> , <code>inverseMethod</code> , <code>cramer</code> ; <i>substitusi mundur</i> dan penentuan jenis solusi.
	<code>SPLResult</code>	Hasil SPL: jenis (<code>UNIQUE</code> , <code>NONE</code> , <code>INFINITE</code>) dan ekspresi tiap variabel $x_i = c + \sum_k a_k t_k$.
<code>determinant</code>	<code>Determinant</code>	<code>byRowReduction</code> (OBE) dan <code>byCofactorExpansion</code> (memilih baris/kolom dengan nol terbanyak).
<code>inverse</code>	<code>Inverse</code>	<code>byGaussJordan</code> (augmentasi $[A \mid I]$) dan <code>byAdjoint</code> .
<code>interpolasi</code>	<code>Interpolasi</code>	<code>calculateCoefficient</code> (SPL Vandermonde), <code>evaluator</code> (skema Horner), <code>toEquation</code> .
	<code>NaturalCubicSpline</code>	Membangun SPL tridiagonal untuk M_i , koefisien a_j, b_j, c_j, d_j , <code>evaluate</code> , <code>toString</code> .
<code>regression</code>	<code>SplineRegression</code>	Basis <i>truncated power</i> , matriks desain X , persamaan normal, <code>predict</code> , <code>toEquation</code> .
<code>io</code>	<code>FileIO</code>	Membaca matriks/titik dari <code>.txt</code> , <code>parseNumber</code> (menerima titik atau koma desimal), menulis hasil.

Paket	Kelas	Tanggung jawab dan metode penting
cli (akar)	ConsoleInput	Masukan <i>keyboard</i> yang tervalidasi (bilangan bulat terbatas, angka, y/n).
	Style, InputEnd-Exception	Tampilan CLI (menu, banner, warna, mode -plain) dan penanda berakhirnya masukan.
	CLI	Menu utama, submenu metode, pembacaan masukan, penyusunan laporan, dan penyimpanan hasil.
	App	Titik masuk program (main).

3.2 Arsitektur Pustaka

Pustaka disusun berlapis: lapisan terbawah adalah Matrix; di atasnya Elimination menyediakan eliminasi baris yang dipakai bersama oleh seluruh modul. Modul interpolasi dan regresi tidak menulis penyelesai sendiri, melainkan menyusun SPL lalu memanggil `SPLSolver.gauss`. Lapisan cli dan io hanya berinteraksi dengan pengguna dan tidak berisi logika matematika.



Gambar 1: Ketergantungan antarmodul (panah: “memakai”). `SPLSolver` memakai `Determinant` (Kaidah Cramer) dan `Inverse` (metode balikan); `Inverse` juga memakai `Determinant` (metode adjoin)

3.3 Rincian Algoritma

Eliminasi dengan *partial pivoting*. Untuk tiap kolom, dicari baris pada/ di bawah baris pivot saat ini yang memiliki $|a_{ik}|$ terbesar lalu ditukar ke posisi pivot. Kolom dilewati jika pivot terbesar tidak lebih dari toleransi $\varepsilon = \max |a_{ij}| \cdot \max(m, n) \cdot 10^{-15}$. Semua elemen di bawah

pivot dinolkan, dan jumlah pertukaran baris dikembalikan agar dapat dipakai menghitung determinan. Gauss-Jordan melanjutkan dengan membagi baris pivot dan menolkan elemen di atas pivot.

Penentuan jenis solusi. Setelah eliminasi, setiap baris diperiksa posisi elemen tak nolnya yang pertama. Bila posisi itu tepat di kolom konstanta, SPL tidak konsisten (NONE). Variabel tanpa pivot menjadi variabel bebas $t_1, t_2, \dots$; variabel berpivot dihitung dari baris terbawah ke atas dalam bentuk $c + \sum a_k t_k$ (substitusi mundur). Solusi tunggal berarti tidak ada variabel bebas. Bentuk parametrik dicetak sebagai, misalnya, $x_1 = 3.9 - 2t_1 - 2.9t_2$.

Determinan. Metode OBE mereduksi salinan matriks ke segitiga atas, lalu mengalikan elemen diagonal dengan $(-1)^s$; bila ada diagonal bernilai nol (dalam toleransi) hasilnya 0. Metode kofaktor bersifat rekursif dan pada tiap level memilih baris atau kolom dengan jumlah nol terbanyak, sehingga suku bernilai nol dilewati; dibatasi hingga 11×11 .

Matriks balikan. Metode augmentasi membentuk $[A \mid I]$ lalu mereduksinya dengan Gauss-Jordan; kesingularan diperiksa lebih dulu melalui reduksi baris. Metode adjoin menghitung seluruh kofaktor (determinan minor dengan reduksi baris), mentranspos menjadi $\text{adj}(A)$, lalu mengalikannya dengan $1/\det(A)$. Metode balikan untuk SPL menghitung $\mathbf{x} = A^{-1}\mathbf{b}$ dan Kaidah Cramer menghitung $\det(A_i)/\det(A)$; keduanya hanya untuk SPL persegi.

Interpolasi polinomial. Matriks Vandermonde disusun dari titik-titik masukan dan diselesaikan dengan `SPLSolver.gauss`; nilai $P(x_t)$ dihitung dengan skema Horner. Evaluasi hanya diizinkan di dalam domain $[x_{\min}, x_{\max}]$.

Natural cubic spline. Titik diurutkan menurut x (pengurutan sisip), lalu h_j dihitung dan SPL tridiagonal berukuran $(n-2) \times (n-2)$ dirakit dan diselesaikan dengan `SPLSolver.gauss`; $M_0 = M_{n-1} = 0$ diterapkan secara langsung. Koefisien tiap segmen dihitung dengan rumus pada Bab 2, dan evaluasi mencari segmen yang memuat x_t .

Regresi spline kubik. Pengguna menentukan jumlah dan posisi *knot*. Program menyusun matriks desain X ($N \times (4 + K)$) dengan *truncated power basis*, menghitung $X^T X$ dan $X^T \mathbf{y}$ memakai operasi `Matrix`, lalu menyelesaikan persamaan normal dengan eliminasi Gauss. Program menolak masukan bila $N \leq 4 + K$ atau bila dua *knot* sama.

3.4 Alur Program

1. `App.main` menginisialisasi `Style` (opsi `-plain` mematikan warna/Unicode) lalu menjalankan `CLI.run`.

2. CLI menampilkan **menu utama** (1. SPL, 2. Determinan, 3. Matriks Balikan, 4. Interpolasi Polinomial, 5. *Natural Cubic Spline*, 6. Regresi Spline Kubik, 7. Keluar). Menu 1, 2, 3, 4, dan 6 memiliki **submenu** pemilihan metode.
3. Pengguna memilih sumber masukan: *keyboard* (ukuran matriks dibatasi 11×11 , atau 11×12 untuk *augmented*; titik data maksimal 10) atau berkas *.txt* (matriks hingga 1001×1001 , atau 1001×1002 untuk *augmented*).
4. Modul pustaka dipanggil dengan *StringBuilder* untuk mencatat langkah perhitungan (OBE, matriks antara, substitusi mundur, kofaktor, dan sebagainya). Untuk matriks yang lebih besar dari batas masukan *keyboard*, langkah tidak dicatat agar keluaran tidak membanjiri layar.
5. Laporan berisi metode, matriks/titik masukan, langkah, dan hasil ditampilkan di layar. Untuk interpolasi, *spline*, dan regresi, program lalu meminta nilai x berulang kali (kosongkan untuk selesai) untuk dievaluasi.
6. Pengguna dapat menyimpan seluruh laporan ke berkas *.txt*; program kembali ke menu utama.

3.5 Penanganan Kesalahan

Setiap kesalahan masukan atau komputasi ditangani dengan mekanisme *exception* Java sehingga program tidak *crash*.

Situasi	Exception	Perilaku program
Karakter bukan angka pada masukan <i>keyboard</i> atau berkas	<code>NumberFormatException</code>	Pesan galat; <i>keyboard</i> : ulangi pembacaan baris
Jumlah elemen baris tidak sama / dimensi tidak cocok	<code>IllegalArgumentException</code>	Pesan galat menyebut nomor baris
Berkas tidak ditemukan atau tidak dapat dibuka	<code>FileNotFoundException</code> , <code>IOException</code>	Pesan galat, kembali ke menu
Berkas kosong, baris kosong, atau melebihi batas ukuran	<code>IllegalArgumentException</code>	Pesan “format isi file tidak sesuai”
Determinan matriks non-persegi; invers singular; Cramer/balikan pada SPL non-persegi atau $\det = 0$	<code>IllegalArgumentException</code> , <code>ArithmeticException</code>	Pesan, mis. “matriks tidak memiliki balikan”
Determinan melebihi jangkauan <code>double</code>	<code>ArithmeticException</code>	Pesan “terlalu besar”
Masukan <i>keyboard</i> berakhir (EOF)	<code>InputException</code>	Program berhenti dengan rapi
Kesalahan tak terduga di modul	<code>RuntimeException</code>	Ditangkap di <code>CLI.run</code> , program lanjut

Tabel 2: Ringkasan penanganan kesalahan

Selain itu, `FileIO.parseNumber` memvalidasi angka dengan ekspresi reguler sehingga tanda desimal titik maupun koma diterima (3,5 dan 3.5), dan seluruh keluaran dibulatkan maksimal 3 angka di belakang koma (`Matrix.round3/formatNumber`) tanpa pemisah ribuan. Nama berkas tidak pernah dipakai untuk menentukan atau memvalidasi jenis masukan.

3.6 Penggunaan Maven dan Pembuatan Berkas JAR

Proyek dikelola dengan Apache Maven. Berkas `pom.xml` mendefinisikan `groupId` `algeo`, `artifactId` `matrix-calculator`, serta `source/target` Java 17 dan penyandian UTF-8. Kelas utama adalah `algeo.App`. Perintah yang dipakai:

```
mvn clean compile      # kompilasi
mvn exec:java          # menjalankan program CLI (exec-maven-plugin, mainClass algeo.App)
mvn clean package      # membuat berkas JAR di direktori target/
```

Agar JAR hasil package dapat dijalankan langsung dengan `java -jar`, `pom.xml` memuat `maven-jar-plugin` dengan entri `manifest Main-Class` bernilai `algeo.App`. Berkas JAR kemudian disalin ke folder `bin/` dan dijalankan dengan:

```
java -jar bin/matrix-calculator-1.0-SNAPSHOT.jar
```

4 Eksperimen

Seluruh kasus uji pada Bab 5 dokumen spesifikasi dijalankan melalui program CLI, baik dengan masukan *keyboard* maupun berkas `.txt` (berkas masukan tersimpan di folder `test/`). Keluaran program yang ditampilkan di sini adalah keluaran asli (tanda desimal titik, pembulatan 3 angka). Untuk analisis, hasil program dibandingkan dengan perhitungan eksak (aritmetika pecahan) yang dihitung terpisah.

4.1 Determinan Matriks (Kasus 5.1)

Setiap matriks diuji dengan reduksi baris (OBE) dan ekspansi kofaktor. Masukan: det_5_1_1.txt sampai det_5_1_3.txt, berisi matriks

$$A_{5.1.1} = \begin{bmatrix} 0 & 2 & -1 & 3 \\ 4 & 1 & 2 & -2 \\ -2 & 3 & 0 & 1 \\ 1 & -1 & 5 & 2 \end{bmatrix}, \quad A_{5.1.2} = \begin{bmatrix} 2 & -1 & 3 & 0 & 4 \\ 1 & 2 & -2 & 5 & -1 \\ 3 & 1 & 1 & 2 & 0 \\ 0 & 4 & -1 & 3 & 2 \\ 5 & 0 & 4 & 2 & 4 \end{bmatrix},$$

$$A_{5.1.3} = \begin{bmatrix} \frac{1}{2} & 1 & 0 & \frac{1}{2} \\ \frac{1}{2} & \frac{5}{4} & \frac{1}{4} & \frac{1}{2} \\ -\frac{1}{10} & -\frac{2}{5} & \frac{1}{5} & -\frac{3}{10} \\ \frac{3}{5} & \frac{5}{4} & \frac{9}{20} & \frac{3}{5} \end{bmatrix}.$$

Elemen pecahan pada kasus 5.1.3 dituliskan dalam berkas sebagai bilangan desimal (0.5, 1.25, -0.1, dan seterusnya).

Kasus	Ukuran / catatan	OBE	Kofaktor	Nilai eksak
5.1.1	4×4 bilangan bulat	-259	-259	-259
5.1.2	5×5 bilangan bulat	0	0	0
5.1.3	4×4 pecahan (dimasukkan sebagai desimal)	0.01	0.01	1/100

Tabel 3: Hasil determinan

Contoh keluaran langkah (kasus 5.1.1, ekspansi kofaktor):

```
Ekspansi kofaktor sepanjang baris 1:
a12 = 2, M12 = 10, C12 = (-1)^3 x M12 = -10
a13 = -1, M13 = 35, C13 = (-1)^4 x M13 = 35
a14 = 3, M14 = 68, C14 = (-1)^5 x M14 = -68
det = jumlah a x C = -259
```

Pada kasus 5.1.1 elemen $a_{11} = 0$ sehingga baris pertama dipilih (memuat nol) dan suku $a_{11}C_{11}$ dilewati. Pada reduksi baris, pivot sebesar 4 dipilih pada kolom pertama (*partial pivoting*) dan terjadi 3 pertukaran baris, sehingga $\det = (-1)^3 \cdot 4 \cdot 3.5 \cdot 4.857 \cdot 3.809 = -259$.

Analisis. Kedua metode menghasilkan nilai yang sama dan sesuai nilai eksak. Pada kasus 5.1.2 matriks singular (baris-barisnya bergantung linier); reduksi baris mengenalinya karena pivot tidak melebihi toleransi ε sehingga menghasilkan tepat 0, bukan nilai sangat kecil seperti 10^{-16} , dan ini penting agar pemeriksaan kebalikan benar. Kasus 5.1.3 memuat elemen pecahan (0.5, 0.25, -0.1, dan seterusnya) yang tidak semuanya dapat dinyatakan tepat dalam biner; hasil 0.01 tetap tepat setelah pembulatan walaupun perhitungan memakai *floating point*.

4.2 Matriks Balikan (Kasus 5.2)

Masukan: `inverse_5_2_1.txt` dan `inverse_5_2_2.txt`, berisi matriks

$$A_{5.2.1} = \begin{bmatrix} 0 & 1 & 2 & -1 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ -1 & 2 & 1 & 0 \end{bmatrix}, \quad A_{5.2.2} = \begin{bmatrix} 1 & 2 & 0 & -1 & 3 \\ 2 & -1 & 4 & 0 & 1 \\ 0 & 3 & 1 & 2 & -2 \\ 4 & 0 & 5 & 1 & 2 \\ 3 & 1 & 4 & -1 & 4 \end{bmatrix}.$$

Kasus 5.2.1 (4×4). Kedua metode (augmentasi $[A \mid I]$ dan adjoin) menghasilkan matriks yang sama:

```
A^-1 =
[ 0.200 -0.100 0.400 -0.300 ]
[ -0.100 -0.200 0.300 0.400 ]
[ 0.400 0.300 -0.200 -0.100 ]
[ -0.300 0.400 -0.100 0.200 ]
```

Hasil ini sama dengan $A^{-1} = \frac{1}{10} \begin{bmatrix} 2 & -1 & 4 & -3 \\ -1 & -2 & 3 & 4 \\ 4 & 3 & -2 & -1 \\ -3 & 4 & -1 & 2 \end{bmatrix}$, konsisten dengan $\det(A) = 20$ yang dihitung secara eksak.

Kasus 5.2.2 (5×5). Kedua metode menampilkan pesan “*Matriks tidak memiliki balikan karena determinannya 0*”. Hal ini sesuai karena $\det(A) = 0$ secara eksak.

Analisis. Kedua metode setara untuk matriks kecil. Metode adjoin memerlukan n^2 determinan minor sehingga jauh lebih mahal daripada Gauss-Jordan untuk n besar, tetapi pada ukuran uji (4×4) perbedaannya tidak terasa. Program menolak matriks singular sebelum eliminasi dimulai sehingga tidak terjadi pembagian dengan nol.

4.3 SPL $Ax = b$ (Kasus 5.3)

Setiap kasus diuji dengan keempat metode. Masukan berupa matriks *augmented* $[A \mid b]$ pada berkas `spl_5_3_1.txt`, `spl_5_3_2.txt`, dan `spl_5_3_3.txt`:

$$5.3.1: \left[\begin{array}{cccc|c} 0 & 2 & -1 & 1 & -5 \\ 1 & -1 & 2 & 3 & 15 \\ 2 & 1 & 1 & -1 & 1 \\ -1 & 3 & 0 & 2 & -3 \end{array} \right], \quad 5.3.2: \left[\begin{array}{ccccc|c} 1 & 2 & -1 & 0 & 3 & 2 \\ 2 & 4 & 1 & -1 & 5 & 10 \\ -1 & -2 & 2 & 3 & -1 & -1 \end{array} \right],$$

$$5.3.3: \left[\begin{array}{cccc|c} 1 & -2 & 3 & 1 & 4 \\ 2 & 1 & -1 & 4 & -1 \\ 0 & 3 & 2 & -2 & 3 \\ 3 & -1 & 2 & 5 & 4 \end{array} \right].$$

Kasus 5.3.4 memakai matriks *discrete Laplacian* A_n (2 pada diagonal utama, -1 tepat di atas dan di bawahnya, 0 lainnya) dengan $b = (1, \dots, 1)^T$ untuk $n = 6$ dan $n = 10$

(spl_5_3_4_n6.txt, spl_5_3_4_n10.txt).

Kasus	Gauss / Gauss-Jordan	Matriks balikan	Kaidah Cramer
5.3.1	$x = (1, -2, 3, 2)$	sama	sama
5.3.2 (3×5)	tak hingga solusi, 2 parameter	ditolak: persamaan (3) $\neq$ variabel (5)	ditolak: persamaan (3) $\neq$ variabel (5)
5.3.3	"Solusi tidak ada"	ditolak: $\det(A) = 0$	ditolak: $\det(A) = 0$
5.3.4, $n = 6$	$x = (3, 5, 6, 6, 5, 3)$	sama	sama
5.3.4, $n = 10$	$x = (5, 9, 12, 14, 15, 15, 14, 12, 9, 5)$	sama	sama

Tabel 4: Ringkasan hasil kasus 5.3

Solusi parametrik kasus 5.3.2 menurut program (Gauss dan Gauss-Jordan sama):

```

x1 = 3.9 - 2t1 - 2.9t2
x2 = t1
x3 = 1.9 + 0.1t2
x4 = -0.3 - 0.7t2
x5 = t2

```

Contoh keluaran langkah Gauss pada kasus 5.3.1:

```

Eliminasi Gauss (menuju matriks eselon baris):
R1 <-> R3
R2 = R2 + (-0.500) R1
R4 = R4 + (0.500) R1
[ 2.000  1.000  1.000 -1.000  1.000 ]
[ 0.000 -1.500  1.500  3.500 14.500 ]
[ 0.000  2.000 -1.000  1.000 -5.000 ]
[ 0.000  3.500  0.500  1.500 -2.500 ]
...
Substitusi mundur:
x4 = 2
x3 = 3
x2 = -2
x1 = 1

```

Analisis. (1) Pada kasus 5.3.1 keempat metode memberi solusi yang sama dengan solusi eksak $(1, -2, 3, 2)$; pivot awal bernilai nol di baris pertama sehingga pertukaran baris oleh *pivoting* wajib terjadi ($R1 \leftrightarrow R3$). (2) Kasus 5.3.2 memiliki rank $3 < 5$ variabel sehingga ada dua variabel bebas (x_2 dan x_5 dilabeli t_1, t_2); metode balikan dan Cramer tidak berlaku karena matriks koefisien tidak persegi, dan program menolaknya dengan pesan yang mengarahkan ke Gauss/Gauss-Jordan. (3) Kasus 5.3.3 memiliki $\text{rank}(A) = 3$ tetapi $\text{rank}([A \mid b]) = 4$ sehingga tidak konsisten; kedua metode eliminasi mendeteksinya melalui baris $0 = c \neq 0$, sedangkan Cramer/balikan ditolak karena $\det(A) = 0$. (4) Kasus 5.3.4 adalah matriks *discrete Laplacian* 1 dimensi dengan solusi analitis $x_i = \frac{i(n+1-i)}{2}$; hasil program untuk $n = 6$ dan $n = 10$ cocok persis dengan rumus ini.

4.4 SPL Berbentuk Matriks *Augmented* (Kasus 5.4)

Masukan (spl_5_4_1.txt dan spl_5_4_2.txt):

$$5.4.1: \left[\begin{array}{cccc|c} 0 & 1 & -1 & 2 & -1 \\ 2 & 0 & 1 & -1 & 3 \\ -1 & 2 & 0 & 1 & -3 \\ 1 & -1 & 2 & 0 & 9 \end{array} \right], \quad 5.4.2: \left[\begin{array}{cccccc|c} 1 & 0 & 2 & -1 & 0 & 3 & -7 \\ 0 & 1 & -1 & 2 & 1 & 0 & 6 \\ 2 & -1 & 0 & 1 & 3 & -2 & 13 \\ 1 & 1 & 1 & 1 & 1 & 1 & 3 \end{array} \right].$$

Kasus	Gauss / Gauss-Jordan	Matriks balikan / Cramer
5.4.1 (4×4)	$x = (1, -2, 3, 2)$	sama
5.4.2 (4×6)	$x_1 = 5.8 - t_1 - 1.6t_2$, $x_2 = 2.6 - t_1 - 0.2t_2$, $x_3 = -3.4 + t_1 + 0.8t_2$, $x_4 = t_1$, $x_5 = t_2$, $x_6 = -2$	ditolak: persamaan (4) $\neq$ variabel (6)

Tabel 5: Hasil kasus 5.4

Analisis. Kasus 5.4.1 ternyata memiliki solusi yang sama dengan kasus 5.3.1 sehingga dapat dipakai sebagai pemeriksaan silang. Kasus 5.4.2 memiliki $x_6 = -2$ yang tetap (bukan variabel bebas) dan dua variabel bebas x_4, x_5 ; hasil sesuai dengan penyelesaian eksak (*linsolve*) yang dihitung terpisah.

4.5 SPL Berbentuk Umum (Kasus 5.5)

Persamaan diubah menjadi matriks *augmented* (variabel yang tidak muncul diberi koefisien 0) dan disimpan pada `sp1_5_5_1.txt` dan `sp1_5_5_2.txt`. Sistem masukannya adalah

$$5.5.1: \begin{cases} 3x_1 + 2x_2 - x_3 + 4x_4 = 5 \\ x_1 - 3x_2 + 2x_3 + x_4 = -2 \\ 4x_1 + x_2 + 5x_3 - 2x_4 = 7 \\ 2x_1 - x_2 + 3x_3 + x_4 = 3 \end{cases}$$

$$5.5.2: \begin{cases} x_1 + x_2 + x_3 = 10.00 \\ x_4 + x_5 + x_6 = 12.00 \\ x_7 + x_8 + x_9 = 9.00 \\ 0.03521(x_1 + x_4 + x_7) + 0.68(x_2 + x_6) + 0.55432x_9 = 8.42 \\ 0.87654(x_1 + x_4 + x_7) + 0.30(x_3 + x_5 + x_8) = 11.05 \\ 0.03521(x_1 + x_4 + x_7) + 0.68(x_3 + x_5) + 0.55432x_2 = 6.19 \\ x_1 + x_5 + x_9 = 11.00 \\ x_2 + x_4 + x_8 = 8.50 \\ x_3 + x_6 + x_7 = 11.50 \end{cases}$$

Kasus 5.5.1 (4×4): solusi tunggal $x_1 = -0.114$, $x_2 = 1.841$, $x_3 = 1.432$, $x_4 = 0.773$ oleh keempat metode. Nilai eksaknya $(-\frac{5}{44}, \frac{81}{44}, \frac{63}{44}, \frac{17}{22})$, sehingga hasil sesuai.

Kasus 5.5.2 (9×9 , koefisien desimal): Gauss dan Gauss-Jordan menghasilkan solusi dengan satu parameter:

$x_1 = 5.781 - 0.462t_1$	$x_6 = 7.506 - 0.433t_1$
$x_2 = 4.399 - 0.387t_1$	$x_7 = 4.174 - 0.416t_1$
$x_3 = -0.18 + 0.849t_1$	$x_8 = 4.826 - 0.584t_1$
$x_4 = -0.725 + 0.971t_1$	$x_9 = t_1$
$x_5 = 5.219 - 0.538t_1$	

Metode matriks balikan dan Kaidah Cramer menampilkan pesan bahwa metode tidak dapat dipakai karena $\det(A) = 0$.

Analisis. Matriks koefisien kasus 5.5.2 singular dengan $\text{rank} = 8$ dari 9 variabel, sehingga ada tepat satu variabel bebas. Karena koefisiennya desimal, pengenalan pivot nol bergantung pada toleransi ε (Bab 3): tanpa toleransi, sisa galat pembulatan dapat membuat baris yang seharusnya nol tampak tak nol dan menghasilkan klaim “solusi tunggal” atau “tidak ada solusi” yang keliru. Hasil program cocok dengan penyelesaian eksak (aritmetika pecahan) pada seluruh koefisien.

4.6 Aplikasi SPL: Regresi Linier Harga Rumah (Kasus 5.6)

Masukan berupa data luas rumah dan harga $(x, y) = (1, 3), (2, 5), (3, 6)$. Persamaan normalnya disusun menjadi matriks *augmented* $\begin{bmatrix} 3 & 6 & 14 \\ 6 & 14 & 31 \end{bmatrix}$ (sp1_5_6.txt), yaitu

$$\begin{aligned} 3\theta_0 + 6\theta_1 &= 14, \\ 6\theta_0 + 14\theta_1 &= 31, \end{aligned}$$

dengan $\sum x_i = 6$, $\sum x_i^2 = 14$, $\sum y_i = 14$, dan $\sum x_i y_i = 31$. Keempat metode menghasilkan $\theta_0 = 1.667$ dan $\theta_1 = 1.5$ (eksak: $\theta_0 = 5/3$, $\theta_1 = 3/2$), yaitu model

$$\hat{y} = 1.667 + 1.5x$$

dengan satuan x dalam ratusan m^2 dan y dalam ratusan juta. Artinya setiap penambahan $100 m^2$ luas rumah menaikkan harga sekitar 150 juta. Karena sistem berukuran 2×2 dan non-singular ($\det = 6$), keempat metode setara.

4.7 Interpolasi Polinomial (Kasus 5.7)

Studi kasus 1 (7 titik). Masukan (interpolasi_5_7a.txt):

x	0.2	0.4	0.6	0.8	1.0	1.2	1.4
$f(x)$	0.182	0.336	0.470	0.588	0.693	0.788	0.875

Polinom derajat 6 yang diperoleh:

$P(x) = 1.003x - 0.546x^2 + 0.479x^3 - 0.408x^4 + 0.208x^5 - 0.043x^6$
Domain interpolasi: $0.2 \leq x \leq 1.4$

x	0.3	0.55	0.95	1.38
$P(x)$ (program)	0.262	0.438	0.668	0.867

Tabel 6: Hasil evaluasi $f(x)$ pada kasus 5.7a (semua titik berada dalam domain)

Nilai-nilai tersebut sama dengan evaluasi polinom eksak (0.26186; 0.43812; 0.66780; 0.86667). Data tampak mengikuti $\ln(1+x)$ (0.2624; 0.4383; 0.6678; 0.8671), dengan selisih di bawah 10^{-3} .

Studi kasus 2 (suhu komponen elektronik, 6 titik). Masukan (interpolasi_5_7b.txt):

t (menit)	0	2	4	6	8	10
T ($^{\circ}C$)	25.0	31.2	38.6	47.5	58.1	70.8

Polinom yang diperoleh:

$$P(x) = 25 + 2.892x + 0.07x^2 + 0.02x^3 - 0.002x^4$$

t (menit)	1	3.5	7	9.2
T (program, °C)	27.981	36.617	52.571	65.425

Tabel 7: Perkiraan suhu pada kasus 5.7b

Waktu komponen rusak ($T = 65^\circ\text{C}$). Program tidak memiliki fitur pencari akar, sehingga dilakukan pemindaian: nilai $P(t)$ dievaluasi untuk $t = 8.00; 8.01; \dots; 10.00$ melalui fitur evaluasi. Diperoleh $P(9.13) = 64.975$ dan $P(9.14) = 65.039$, sehingga dengan interpolasi linier $t \approx 9.134$ menit, atau sekitar 9 menit 8 detik. Pencarian akar eksak $P(t) = 65$ memberi $t = 9.1339$, sehingga hasil pemindaian akurat hingga tiga angka desimal.

Analisis. (1) Nilai evaluasi program sama dengan perhitungan eksak. (2) Persamaan yang *dicetak* dibulatkan 3 angka desimal, sedangkan evaluasi memakai koefisien *double* penuh. Akibatnya, mengevaluasi persamaan cetak dengan tangan memberi hasil yang berbeda jauh: koefisien eksak $a_4 = -0.0018229$ tercetak -0.002 dan $a_5 = 7.8 \times 10^{-5}$ tercetak 0 (dibuang), sehingga $P(9.2)$ dari persamaan cetak adalah 58.777, padahal nilai sebenarnya 65.425. Karena itu nilai $P(x_t)$ yang dilaporkan program lebih dapat dipercaya daripada persamaan cetak, dan pembulatan 3 angka (sesuai spesifikasi) sebaiknya tidak dipakai untuk menghitung ulang. (3) Polinom derajat 5 untuk data yang hampir kuadratik–kubik masih berperilaku mulus pada rentang data, tetapi derajat tinggi dengan titik berjarak seragam rentan berosilasi di dekat ujung interval; hal ini tidak terlihat di sini karena datanya halus dan monoton.

4.8 Natural Cubic Spline (Kasus 5.8)

Lima titik $(1,2), (2,3), (3,4), (4,5), (5,6)$ (masukan `interpolasi_5_8.txt`). Program membangun SPL tridiagonal berukuran 3×3 :

```
SPL tridiagonal untuk M1..M3 (M0 = M4 = 0):
[ 4.000  1.000  0.000  0.000 ]
[ 1.000  4.000  1.000  0.000 ]
[ 0.000  1.000  4.000  0.000 ]
```

Hasilnya $M_0 = \dots = M_4 = 0$ dan keempat segmen berbentuk garis lurus:

```
S0(x) = 2 + (x - 1), untuk 1 <= x <= 2    S2(x) = 4 + (x - 3), untuk 3 <= x <= 4
S1(x) = 3 + (x - 2), untuk 2 <= x <= 3    S3(x) = 5 + (x - 4), untuk 4 <= x <= 5
```

Evaluasi: $S(1.5) = 2.5$, $S(2.5) = 3.5$, $S(3.5) = 4.5$, $S(4.5) = 5.5$.

Analisis. Seluruh titik terletak pada garis $y = x + 1$. Ruas kanan SPL tridiagonal adalah $6\left(\frac{y_{i+1}-y_i}{h_i} - \frac{y_i-y_{i-1}}{h_{i-1}}\right) = 0$ sehingga $M_i = 0$ (sistem homogen dengan matriks non-singular), dan setiap segmen tereduksi menjadi $S_j(x) = x + 1$. Ini sesuai sifat teoretis *natural cubic spline*: spline mereproduksi data linier secara tepat, tanpa osilasi, berbeda dengan interpolasi polinomial tunggal yang tidak dijamin demikian untuk sembarang data.

4.9 Regresi Spline (Kasus 5.9)

Masukan berupa 7 titik pengamatan pada berkas `regresi_5_9.txt` (ditulis dengan tanda desimal koma seperti pada spesifikasi, misalnya 19,9) dan satu *knot* internal $\xi_1 = 0$ yang dimasukkan melalui *keyboard* (jumlah *knot* $K = 1$, posisi 0):

x_i	-3	-2	-1	0	1	2	3
y_i	19.9	4.4	-0.5	2	6.5	15.6	38.1

Dengan $K = 1$, ruang *spline* kubik berdimensi $M = 4 + 1 = 5$ dengan basis $\{1, x, x^2, x^3, (x - 0)_+^3\}$. Karena $N = 7 > M = 5$, sistem $X\beta \approx \mathbf{y}$ bersifat *overdetermined* dan diselesaikan melalui persamaan normal. Matriks desain dan persamaan normal yang dibentuk program:

```

Matriks desain X:
[ 1.000 -3.000  9.000 -27.000  0.000 ]
[ 1.000 -2.000  4.000 -8.000  0.000 ]
[ 1.000 -1.000  1.000 -1.000  0.000 ]
[ 1.000  0.000  0.000  0.000  0.000 ]
[ 1.000  1.000  1.000  1.000  1.000 ]
[ 1.000  2.000  4.000  8.000  8.000 ]
[ 1.000  3.000  9.000  27.000 27.000 ]

Persamaan normal [X^T X | X^T y]:
[ 7.000  0.000 28.000  0.000 36.000 86.000 ]
[ 0.000 28.000  0.000 196.000 98.000 84.000 ]
[ 28.000  0.000 196.000  0.000 276.000 608.000 ]
[ 0.000 196.000  0.000 1588.000 794.000 588.000 ]
[ 36.000 98.000 276.000 794.000 794.000 1160.000 ]

```

SPL 5×5 tersebut diselesaikan dengan eliminasi Gauss berpivot (terjadi pertukaran $R1 \leftrightarrow R5$, $R2 \leftrightarrow R4$, dan $R3 \leftrightarrow R5$), lalu substitusi mundur. Keluaran program:

```

Koefisien regresi:
beta0 = 2
beta1 = 3
beta2 = 0
beta3 = -1
beta4 = 2
y(x) = 2 + 3x - x^3 + 2(x - 0)^3_+

```

Jadi model regresinya

$$\hat{y}(x) = 2 + 3x - x^3 + 2(x)_+^3 = \begin{cases} 2 + 3x - x^3, & x \leq 0, \\ 2 + 3x + x^3, & x > 0. \end{cases}$$

Hasil prediksi program pada titik data beserta residunya:

x	-3	-2	-1	0	1	2	3
y	19.9	4.4	-0.5	2	6.5	15.6	38.1
$\hat{y}(x)$ (program)	20	4	0	2	6	16	38
$e = y - \hat{y}$	-0.1	0.4	-0.5	0	0.5	-0.4	0.1

Tabel 8: Prediksi dan residu regresi *spline* kubik kasus 5.9

Sebagai contoh prediksi di luar titik data, program memberikan $\hat{y}(-2.5) = 10.125$ dan $\hat{y}(0.5) = 3.625$.

Analisis. (1) Penyelesaian persamaan normal secara eksak (aritmetika pecahan) memberikan $\hat{\beta} = (2, 3, 0, -1, 2)$, sama persis dengan keluaran program. (2) Kedua potongan kubik bertemu secara mulus di *knot*: untuk kedua sisi berlaku $\hat{y}(0) = 2$, $\hat{y}'(0) = 3$, dan $\hat{y}''(0) = 0$, sedangkan turunan ketiga melompat dari -6 menjadi 6 (selisih $6\beta_4 = 12$). Ini sesuai sifat *truncated power basis* yang menjamin kontinuitas C^2 tanpa persamaan batas tambahan. (3) Residu memenuhi syarat ortogonalitas $X^T \mathbf{e} = \mathbf{0}$, misalnya $\sum e_i = 0$ dan $\sum x_i e_i = 0$, sesuai interpretasi kuadrat terkecil sebagai proyeksi ortogonal $\mathbf{y}$ ke $\text{Col}(X)$. Jumlah kuadrat residunya $\sum e_i^2 = 0.84$. (4) Sebagai pembanding, regresi tanpa *knot* ($K = 0$, kubik global) menghasilkan $\hat{y}(x) = -0.286 + 3x + 3.143x^2$ dengan $\sum e_i^2 \approx 18.554$. Penambahan satu *knot* di $x = 0$ menurunkan galat lebih dari 20 kali karena data berperilaku berbeda di kiri dan kanan $x = 0$: di kiri data turun hingga minimum di sekitar $x = -1$, sedangkan di kanan data naik semakin tajam. Perilaku ini tidak dapat ditangkap oleh satu polinom kubik global. (5) Berbeda dengan interpolasi, kurva regresi tidak melalui setiap titik; residu kecil yang tersisa dapat dipandang sebagai derau pengukuran yang diredam oleh model.

4.10 Eksperimen Tambahan

Matriks berukuran besar dari berkas (1001×1001). Matriks acak dominan diagonal ukuran 1001×1002 (*augmented*, `spl_acak_1001.txt`) dan bagian koefisiennya (1001×1001 , `det_acak_1001.txt`) dibaca dari berkas. Waktu total dari peluncuran program (termasuk JVM dan pembacaan berkas) tercatat:

Operasi	Waktu	Hasil
SPL, Gauss (1001×1002)	≈ 1.8 s	solusi tunggal
SPL, Gauss-Jordan (1001×1002)	≈ 2.1 s	solusi tunggal (sama dengan Gauss)
SPL, matriks balikan (1001×1002)	≈ 3.5 s	solusi tunggal (sama dengan Gauss)
Determinan OBE, matriks acak 1001×1001	≈ 1.9 s	pesan “nilai determinan terlalu besar”
Determinan kofaktor 1001×1001	≈ 1.3 s	pesan: hanya hingga 11×11

Tabel 9: Uji batas ukuran masukan berkas

Gauss-Jordan sedikit lebih lambat karena melakukan eliminasi tambahan di atas pivot (sekitar $n^3/2$ operasi dibanding $n^3/3$ untuk Gauss maju). Determinan acak ukuran 1001 melampaui jangkauan `double`; program mendeteksinya (`Double.isInfinite`) dan memberi pesan alih-alih mengembalikan `Infinity`. Metode matriks balikan paling lambat karena harus mereduksi $[A \mid I]$ berukuran 1001×2002 sebelum mengalikan $A^{-1}\mathbf{b}$. Ekspansi kofaktor ditolak karena kompleksitasnya faktorial; waktu ≈ 1.3 s pada baris tersebut seluruhnya berasal dari peluncuran JVM dan pembacaan berkas.

Matriks Hilbert (berkondisi buruk). Dengan $A_{ij} = 1/(i+j-1)$ dan $\mathbf{b} = A\mathbf{1}$, solusi eksak adalah $x_i = 1$ untuk semua i . Masukan: `spl_hilbert_n6.txt` dan `spl_hilbert_n10.txt`

(elemen ditulis dengan presisi penuh double).

n	Gauss / Gauss-Jordan / Matriks balikan Cramer
6	seluruh $x_i = 1$
10	$x_7 = 1.001, x_8 = 0.999$, lainnya 1
	$x_2 = 1.004, x_3 = 0.986, x_4 = 1.018, x_5 = 0.99, x_6 = 1.002, x_7 = 0.999$, lainnya 1

Tabel 10: Solusi SPL Hilbert

Pada $n = 10$ (bilangan kondisi sekitar 10^{13}) galat membesar. Eliminasi dengan *partial pivoting* menghasilkan galat maksimum sekitar 10^{-3} , sedangkan metode matriks balikan memiliki galat hingga sekitar 2×10^{-2} , yaitu beberapa kali lebih besar karena galat pembulatan menumpuk saat membentuk A^{-1} secara eksplisit lalu mengalikannya dengan $\mathbf{b}$. Hasil ini mendukung keputusan penggunaan *partial pivoting* pada Gauss dan Gauss-Jordan serta menunjukkan bahwa Kaidah Cramer pada implementasi ini cukup stabil karena determinannya dihitung dengan reduksi baris berpivot, bukan dengan ekspansi kofaktor.

5 Penutup

5.1 Kesimpulan

Kami telah membangun pustaka aljabar linier mandiri dalam Java beserta program CLI yang memanfaatkannya. Seluruh fitur wajib tersedia: SPL (Gauss, Gauss-Jordan, matriks balikan, Cramer), determinan (reduksi baris dan kofaktor), matriks balikan (augmentasi dan adjoin), interpolasi polinomial, *natural cubic spline interpolation*, dan regresi spline kubik. Program menampilkan langkah perhitungan, menerima masukan dari *keyboard* dan berkas .txt, menyimpan hasil ke berkas, dan menangani kesalahan melalui *exception*.

Seluruh kasus uji Bab 5 spesifikasi memberikan hasil yang sesuai dengan perhitungan eksak, termasuk regresi *spline* kubik kasus 5.9 yang menghasilkan koefisien $\hat{\beta} = (2, 3, 0, -1, 2)$ secara tepat. Metode yang tidak berlaku untuk suatu kasus (misalnya Kaidah Cramer pada SPL non-persegi atau singular) ditolak dengan pesan yang sesuai.

Kelebihan.

- Desain modular: interpolasi dan regresi memakai kembali SPLSolver; Matrix dan Elimination dipakai bersama.
- *Partial pivoting* dan toleransi nol berbasis skala matriks membuat pengenalan matriks singular dan SPL tak konsisten andal, serta mengurangi galat pada matriks berkondisi buruk (uji Hilbert).
- Batas ukuran masukan (keyboard 11×11 , berkas 1001×1001) dipenuhi, dengan waktu sekitar 2 detik untuk 1001×1002 .

- Pesan kesalahan informatif yang mengarahkan pengguna ke metode yang sesuai (mis. ketika Cramer tidak dapat dipakai).

Kekurangan.

- Fitur bonus antarmuka grafis dan *image inpainting* tidak diimplementasikan; program hanya berbasis CLI (meskipun `pom.xml` sudah memuat dependensi JavaFX dari template).
- Persamaan hasil interpolasi dan regresi dicetak dengan koefisien dibulatkan 3 angka desimal sehingga tidak dapat dipakai menghitung ulang secara manual dengan akurat (Bab 4, kasus 5.7b); nilai evaluasi oleh program tidak terpengaruh.
- SPL tridiagonal *spline* diselesaikan dengan eliminasi Gauss umum, bukan algoritma Thomas $O(n)$; pada $n \leq 10$ perbedaannya tidak berarti.
- Regresi hanya mendukung *spline* kubik, dan posisi *knot* di luar rentang data tidak divalidasi secara eksplisit (kasus demikian akan terdeteksi hanya jika $X^T X$ menjadi singular).
- Metode adjoin dan Cramer berbiaya tinggi untuk n besar; kofaktor dibatasi 11×11 .

5.2 Saran

- Menambahkan antarmuka grafis (JavaFX/Swing) untuk visualisasi OBE dan grafik kurva interpolasi atau regresi.
- Menambahkan modul *image inpainting* berbasis persamaan Laplace dengan metode iteratif (Jacobi atau Gauss-Seidel), yang cocok untuk sistem berukuran besar dan jarang.
- Menampilkan koefisien dengan angka signifikan lebih banyak pada persamaan hasil, atau menyediakan opsi presisi.
- Menambahkan pencarian akar/invers interpolasi (mis. menentukan t untuk T tertentu) dan regresi *spline* berderajat sembarang.
- Menambahkan pengujian otomatis (JUnit) untuk setiap modul agar regresi perilaku mudah dideteksi.

5.3 Refleksi Anggota Kelompok

Jeremy Gerald Sutanto (13525104). Saya merasa tugas besar ini sangat menantang karena harus mengimplementasikan banyak metode aljabar linier dan memastikan hasilnya benar. Saya belajar banyak tentang pengelolaan galat numerik serta *spline* kubik yang baru pertama kali saya dengar. Saya juga merasa tenggat tugas besar ini cukup singkat untuk spesifikasi yang lumayan banyak. Saya beri rating 7/10.

Christian Immanuel (13525116). Refleksi saya dari tugas besar ini, saya belajar implementasi nyata dari penyelesaian SPL dengan metode eliminasi Gauss, Gauss-Jordan, dan Cramer, serta implementasi interpolasi dan *natural cubic spline* ke dalam pemrograman bahasa Java. Melalui tugas besar ini, saya menjadi lebih memahami hubungan antara teori yang dipelajari di perkuliahan dengan penerapannya dalam pemrograman. Selain itu, saya juga belajar membuat CLI sebagai UI yang sederhana. Meskipun begitu, pengerjaan tugas besar ini membutuhkan adaptasi lagi karena saya harus belajar bahasa Java dan dasar-dasarnya lagi dan memiliki deadline yang lumayan singkat. Rating : 8/10

Denzel Santoso (13525014). Menurut saya, tugas besar ini dapat meningkatkan pemahaman saya mengenai materi Aljabar Linier dan Geometri yang diberikan, terutama mengenai determinan dan implementasi OBE dalam eliminasi Gauss, Gauss-Jordan, dan berbagai metode lainnya. Selain itu, kami juga belajar membuat kalkulator tersebut dengan bahasa Java sehingga dapat meningkatkan kemampuan Java walaupun belum pernah diajarkan sebelumnya. Jujur, tenggatnya menurut saya mepet karena bertepatan dengan minggu-minggu yang padat dengan banyak tugas besar lain. Rating 6.7/10.

Pustaka

- [1] H. Anton and C. Rorres, *Elementary Linear Algebra: Applications Version*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2010.
- [2] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Boston, MA, USA: Brooks/Cole, Cengage Learning, 2011, ch. 3.
- [3] C. de Boor, *A Practical Guide to Splines*, rev. ed. New York, NY, USA: Springer-Verlag, 2001.
- [4] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009, ch. 5.
- [5] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: with Applications in R*, 2nd ed. New York, NY, USA: Springer, 2021, ch. 7.

Lampiran

1. **Repositori GitHub kelompok:** <https://github.com/i-Jer/Tubes1-Algeo-2026>
2. **Berkas L^AT_EX:** laporan ini ditulis dengan L^AT_EX; berkas sumber (.tex) beserta asetnya berada di folder docs/ pada repositori (docs/laporan.tex).
3. **Berkas kasus uji:** folder test/ pada repositori memuat berkas masukan .txt untuk seluruh kasus pada Bab 4, termasuk eksperimen tambahan (matriks Hilbert dan matriks acak 1001×1001). Daftar lengkap berkas beserta kasusnya tercantum pada README.md.

4. Pernyataan integritas akademik:

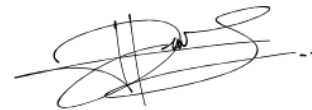
Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Jeremy Gerald Sutanto



Christian Immanuel



Denzel Santoso